Uni-FACEF CENTRO UNIVERSITÁRIO MUNICIPAL DE FRANCA

GUILHERME DE OLIVEIRA PARREIRA
LUCAS ANTONIO MONTEIRO
MATHEUS LOMBARDI DE SOUZA
RAFAEL MENDES FERREIRA

CONSTRUÇÃO DE SOFTWARE ORIENTADO A OBJETOS PARA UMA COMUNIDADE

UCE – ENGENHARIA DE SOFTWARE II

FRANCA
2026

GUILHERME DE OLIVEIRA PARREIRA
LUCAS ANTONIO MONTEIRO
MATHEUS LOMBARDI DE SOUZA
RAFAEL MENDES FERREIRA

CONSTRUÇÃO DE SOFTWARE ORIENTADO A OBJETOS PARA UMA COMUNIDADE

UCE – ENGENHARIA DE SOFTWARE II

Relatório Parcial de Unidade Curricular de Extensão – UCE de Engenharia de Software II, apresentado ao Departamento de Computação do *Uni-FACEF Centro Universitário Municipal de Franca*, para atender às disposições da Resolução nº 7 de 18 de dezembro de 2018, que estabelece as Diretrizes para a Extensão na Educação Superior.

Prof. Responsável: Prof. Me. Carlos Alberto Lucas

FRANCA
2026

1 INTRODUÇÃO

Apresenta-se, neste relatório, o desenvolvimento de uma solução de software no âmbito da Unidade Curricular de Extensão (UCE) intitulada “Construção de Software Orientado a Objetos para uma Comunidade Externa”, vinculada ao Uni-FACEF.

O leitor encontrará nas seções subsequentes o detalhamento dos objetivos do projeto, a metodologia aplicada, o mapeamento dos processos por meio de entrevistas, o Diagrama de Classes UML e o planejamento para a implementação técnica.

A necessidade deste software originou-se da análise do fluxo de trabalho do personal trainer Tiago, que enfrenta desafios críticos na gestão de sua consultoria, contemplando o online e o presencial.

Atualmente, Tiago utiliza ferramentas fragmentadas, como Google Sheets, Excel e o aplicativo MFIT Personal, o que resulta em um "ar de amadorismo" e em uma grande dificuldade de conciliação entre alunos presenciais e online.

O problema central reside na ineficiência operacional: o profissional leva cerca de uma hora para montar um único treino semanal para um único aluno, além de possuir uma gestão financeira quase inexistente, o que gera riscos constantes de inadimplência.

O desafio, portanto, é criar uma ferramenta centralizada e multiplataforma que automatize a prescrição de treinos e profissionalize o controle de clientes e pagamentos.

Alinhado às diretrizes da unidade de extensão, este projeto correlaciona-se diretamente com os Objetivos de Desenvolvimento Sustentável (ODS) da ONU:

- ODS 8 (Trabalho Decente e Crescimento Econômico): Ao otimizar a gestão de tempo e permitir a escalabilidade do atendimento, o software promove a eficiência produtiva do profissional autônomo, apoiando o crescimento de seu microempreendimento.
- ODS 9 (Indústria, Inovação e Infraestrutura): A aplicação de conceitos avançados de Programação Orientada a Objetos para

resolver problemas de uma empresa local representa o fomento à inovação tecnológica e ao fortalecimento da infraestrutura digital.

- ODS 11 (Cidades e Comunidades Sustentáveis): O apoio tecnológico a pequenos negócios da comunidade local contribui para a sustentabilidade econômica e o desenvolvimento da região.

2 PROJETO DE UNIDADE CURRICULAR DE EXTENSÃO - UCE

2.1 OBJETIVO GERAL

O objetivo geral é capacitar o estudante a entender e aplicar os conceitos e fundamentos dos paradigmas de programação e da engenharia de software, para criar aplicações de software que apoiem as atividades de uma pequena ou média empresa.

2.1.1 Objetivos específicos

Apoiar pequenas e médias empresas por meio do desenvolvimento de softwares orientados a objetos.

Automatizar a prescrição de treinos e profissionalizar o controle de clientes para o personal trainer Tiago.

Reduzir o tempo gasto na criação de planilhas de treino e melhorar a visibilidade da saúde financeira do negócio

2.2 METODOLOGIA

A metodologia adotada para este projeto fundamenta-se na aprendizagem baseada em projetos, integrando a teoria de Programação Orientada a Objetos com a prática extensionista junto à comunidade. O processo é estruturado nos seguintes eixos:

Fundamentação Teórica e Conteúdo de Sala de Aula, tendo como base técnica do projeto proveniente das aulas expositivas e práticas em laboratório das disciplinas de Paradigmas de Programação I e Engenharia de Software II. O desenvolvimento do software utiliza o paradigma de Orientação a Objetos (OO), aplicando conceitos fundamentais apresentados pelo docente, tais como:

- Visibilidade de variáveis e métodos.
- Relações de agregação e composição para estruturar os dados do sistema.

- Aplicação de herança e polimorfismo para a criação de categorias de alunos e treinos.
- Uso de classes abstratas e interfaces para garantir a escalabilidade do código.
- Modelagem técnica via Diagrama de Classes da UML para visualização da arquitetura do sistema.

Ida ao Campo e Coleta de Dados, em que a coleta de dados foi realizada por meio de interação e socialização direta (entrevistas) com o cliente, o personal trainer Tiago, de forma online. Este processo permitiu identificar as regras de negócio e as dificuldades operacionais, como a falta de controle financeiro e o tempo excessivo para a prescrição de treinos semanais. As informações coletadas foram processadas utilizando a abordagem do Duplo Diamante, focando na definição de requisitos funcionais que atendam às necessidades reais do profissional.

Instrumentos de Tratamento e Análise, no qual, para a organização das demandas e planejamento da solução, foram empregadas ferramentas estratégicas discutidas em aula:

- Matriz SWOT: Utilizada para mapear forças (conhecimento técnico) e fraquezas (gerenciamento de tempo e pagamentos) do modelo atual de trabalho do Tiago.
- Plano de Ação 2W1H: Aplicado para definir as soluções técnicas prioritárias, como a automação da escrita dos treinos e a criação de uma interface de gestão de alunos e pagamentos.

O projeto segue um cronograma modular, onde cada conceito de OO aprendido em sala é imediatamente aplicado na evolução do código do software.

3 ATORES E ENTREVISTAS COM A EMPRESA

3.1 Caracterização da Empresa e Atores

A "empresa" em questão é o empreendimento de serviços profissionais gerido pelo personal trainer Tiago, que atua de forma autônoma no setor de saúde e bem-estar. O negócio caracteriza-se por um modelo de consultoria híbrida, atendendo tanto alunos de forma presencial quanto de forma online.

Os atores identificados são:

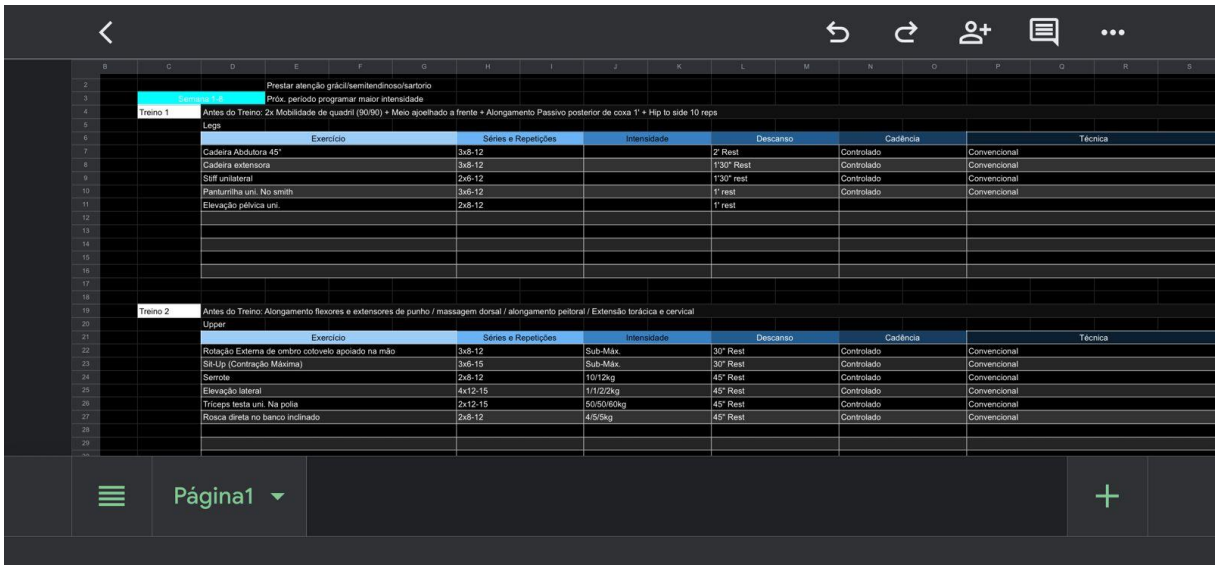
- Tiago (Proprietário/Usuário Principal): Responsável pela prescrição de treinos, avaliação de alunos e gestão financeira do negócio. Suas principais "dores" incluem a fragmentação de dados e a perda de tempo operacional.
- Alunos Presenciais: Clientes que demandam agendamento de horários, local físico de treino e acompanhamento direto.
- Alunos Online: Clientes que utilizam plataformas digitais para receber treinos e enviar feedbacks semanais ou vídeos de execução.
- Equipe de Desenvolvimento (Discentes): Lucas Antonio Monteiro, Matheus Lombardi de Souza e Rafael Mendes Ferreira, responsáveis por converter as necessidades do Tiago em uma solução tecnológica.

3.2 Coleta de Dados e Entrevistas

Diferente das visitas presenciais padrão, a coleta de dados para este projeto foi realizada de forma totalmente online, por meio de contato direto por mensagens (utilizando o WhatsApp) e videochamadas (utilizando o google Meet) com o cliente Tiago. Estes métodos de comunicação, que já são os canais oficiais utilizado por ele para falar com seus alunos, permitiu um registro fiel das suas rotinas e dificuldades.

Através dessas interações digitais, foi possível identificar que o uso de ferramentas como Google Sheets e o app MFIT Personal gera um "ar de

amadorismo" e que a gestão de pagamentos é praticamente inexistente, ocorrendo em planilhas separadas de difícil controle. O cliente manifestou a necessidade de uma ferramenta que centralize as variáveis de treino (volume, intensidade, séries) e forneça relatórios de situação financeira, além do conrole total dos alunos que possui.



		Prestar atenção gráciil/semitendinoso/sartorio				
	Semana 1-4	Próx. período programar maior intensidade				
Treino 1	Antes do Treino: 2x Mobilidade de quadril (90/90) + Meio ajoelhado a frente + Alongamento Passivo posterior de coxa 1' + Hip to side 10 reps					
	Legs					
	Exercício	Séries e Repetições	Intensidade	Descanso	Cadência	Técnica
	Cadeira Abdutora 45°	3x8-12		2' Rest	Controlado	Convencional
	Cadeira extensora	3x8-12		1'30" Rest	Controlado	Convencional
	Stiff unilateral	2x6-12		1'30" rest	Controlado	Convencional
	Panturrilha uni. No smith	3x6-12		1' rest	Controlado	Convencional
	Elevação pélvica uni.	2x8-12		1' rest		
Treino 2	Antes do Treino: Alongamento flexores e extensores de punho / massagem dorsal / alongamento peitoral / Extensão torácica e cervical					
	Upper					
	Exercício	Séries e Repetições	Intensidade	Descanso	Cadência	Técnica
	Rotação Externa de ombro cotovelo apoiado na mão	3x8-12	Sub-Máx.	30" Rest	Controlado	Convencional
	Sit-Up (Contração Máxima)	3x6-15	Sub-Máx.	30" Rest	Controlado	Convencional
	Serrote	2x8-12	10/12kg	45" Rest	Controlado	Convencional
	Elevação lateral	4x12-15	1/1/2/2kg	45" Rest	Controlado	Convencional
	Tríceps testa uni. Na polia	2x12-15	50/50/60kg	45" Rest	Controlado	Convencional
	Rosca direta no banco inclinado	2x8-12	4/5/5kg	45" Rest	Controlado	Convencional

Página1

Figura 1 – Planilha atualmente usada por Tiago.
Fonte: Fornecida pelo cliente (2026)

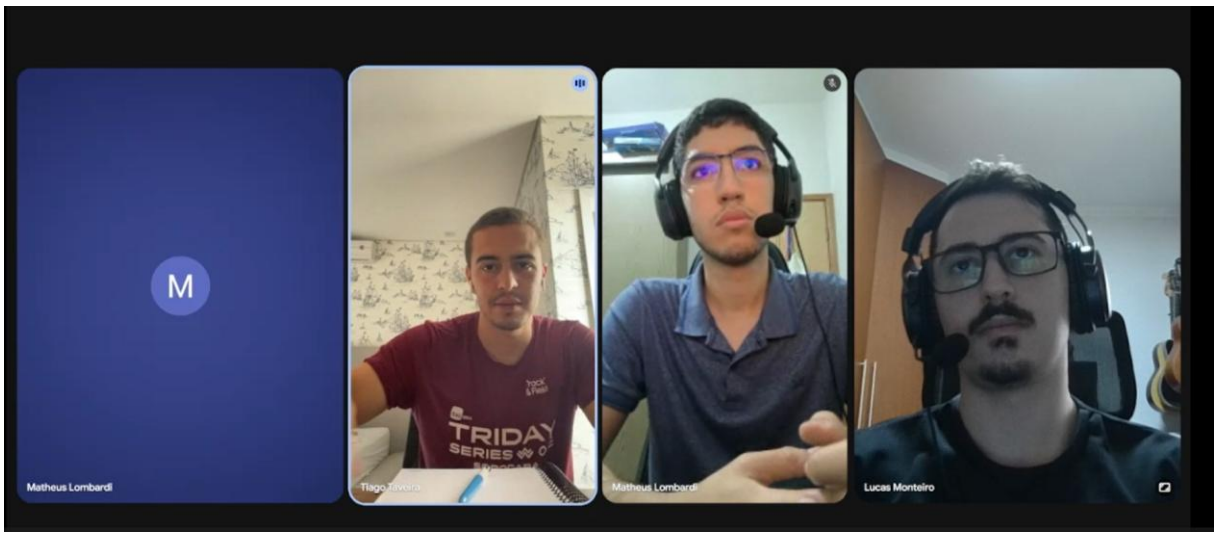


Figura 2 – Entrevista online (Google Meet)
Fonte: Elaborado pelos autores (2026)

4. DIAGRAMA DE CLASSE DA UML

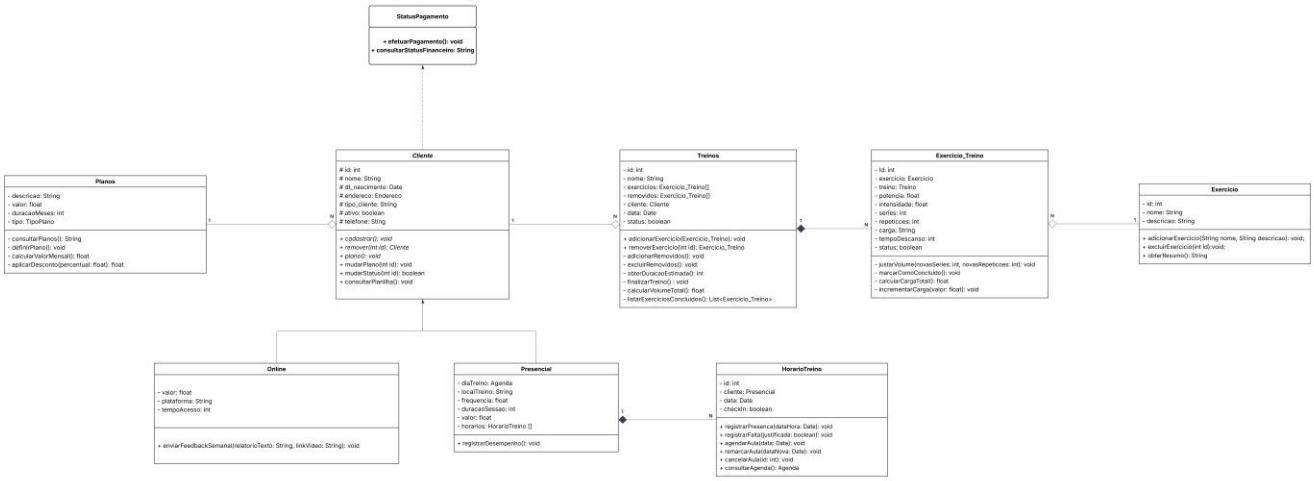


Figura 3 – Diagrama de Classe da UML
Fonte: Elaborada pelos autores (2026).

4.1 Descrição das Classes, Atributos e Métodos

Abaixo, detalhamos as principais classes que compõem o sistema, seus modificadores de acesso e responsabilidades:

- Cliente: Classe Abstrata base que centraliza os dados cadastrais dos alunos.
  - Variáveis (Protegidas #): id, nome, dt_nascimento, endereco, tipo_cliente, ativo e telefone.
  - Métodos (Públicos +): cadastrar() (abstrata), remover() (abstrata), plano() (abstrata), mudarPlano(), mudarStatus() e consultarPlanilha().
- Online (Subclasse de Cliente): Especializada no atendimento remoto.
  - Variáveis (Privadas -): valor, plataforma e tempoAcesso.
  - Métodos (Públicos +): enviarFeedbackSemanal(relatorioTexto, linkVideo), atendendo à necessidade de validar a execução via vídeos.
- Presencial (Subclasse de Cliente): Especializada no atendimento físico em academias.
  - Variáveis (Privadas -): diaTreino (tipo Agenda), localTreino, frequencia, duracaoSessao, valor e um array de horarios.
  - Métodos (Públicos +): registrarDesempenho().

- Treinos: Responsável por gerenciar a prescrição semanal para reduzir o tempo gasto pelo Tiago.
    - Variáveis (Privadas -): id, nome, data, cliente, status e listas de exercicios e removidos.
    - Métodos (Públicos +): adicionarExercicio() e removerExercicio(). Possui também métodos privados (-) para uso interno, como calcularVolumeTotal(), obterDuracaoEstimada() e finalizarTreino().
- Exercicio: Cadastro técnico dos movimentos.
    - Variáveis (Privadas -): id, nome e descricao.
    - Métodos (Públicos +): adicionarExercicio(), excluirExercicio() e obterResumo().
- Exercicio_Treino: Classe de associação entre as classes Treino e Exercicio.
    - Variáveis (Privadas -): id, exercício, treino, potencia, intensidade, series, repeticoes, carga, tempoDescanso e status.
    - Métodos (Privados -): ajustarVolume(), marcarComoConcluido() e incrementarCarga().
- Planos: Gerencia a parte comercial e financeira para evitar a inadimplência.
    - Variáveis (Privadas -): descricao, valor, duracaoMeses e tipo.
    - Métodos (Privados -): consultarPlanos(), definirPlano(), calcularValorMensal() e aplicarDesconto().
- HorarioTreino: Responsável por gerenciar a agenda específica de atendimentos presenciais e o controle de assiduidade.
    - Variáveis (Privadas -): id (int), cliente (do tipo Presencial), data (Date) e checkIn (boolean).
    - Métodos (Públicos +): registrarPresenca(dataHora), registrarFalta(justificada), agendarAula(data), remarcarAula(dataNova), cancelarAula(id) e consultarAgenda().

4.2 Relacionamentos: Herança, Agregação e Composição

O diagrama utiliza os pilares da orientação a objetos para estruturar as regras de negócio:

- Herança: As classes Online e Presencial herdam da classe Cliente. Isso permite que ambas compartilhem os atributos básicos, mas possuam comportamentos específicos, como o envio de vídeos para alunos online ou o agendamento de horários para presenciais.
- Agregação: A classe Treinos possui uma relação de agregação com Exercicio_Treino (representada pela multiplicidade 1 para N), indicando que um treino é composto por vários exercícios, mas os exercícios existem independentemente do treino no banco de dados.

- Composição: A relação entre Presencial e HorarioTreino é de composição, onde os horários agendados dependem diretamente da existência do vínculo do aluno com o personal para serem válidos na agenda.
- Associação: Existe uma associação entre Exercicio_Treino, Exercicio e Treino, permitindo que o sistema vincule uma ficha técnica de exercício a uma prescrição específica com variáveis de carga e repetições personalizadas.

4.3 Classes Abstratas e Interfaces

A classe Cliente é projetada como abstrata, pois no contexto do negócio do Tiago, não existem "clientes genéricos"; todo aluno deve ser obrigatoriamente classificado como Online ou Presencial para que o sistema saiba quais funcionalidades liberar.

A classe StatusPagamento funciona como uma interface de contrato financeiro, contendo os métodos públicos efetuarPagamento() e consultarStatusFinanceiro(), que podem ser implementados por diferentes tipos de planos ou métodos de pagamento futuros.

5 IMPLEMENTAÇÃO DO DIAGRAMA DE CLASSE

5.1 Implementação de Interface e Classe Abstrata

Para garantir que o sistema seja organizado e escalável, utilizamos dois conceitos fundamentais:

- Classe Interface (StatusPagamento): Funciona como um "contrato". Ela obriga qualquer classe que a implemente a ter métodos de pagamento, garantindo que o controle de inadimplência seja padronizado.
- Classe Abstrata (Cliente): Define a base para todos os alunos, mas impede a criação de um cliente "genérico". No negócio do Tiago, o aluno deve ser obrigatoriamente Online ou Presencial.

```
1  public interface StatusPagamento {  no usages
2
3      void efetuarPagamento();  no usages
4
5      String consultarStatusFinanceiro();  no usages
6  }
```

Figura 4 – Interface StatusPagamento().
Fonte: Elaborado pelos autores (2026)

```java
public abstract class Cliente implements StatusPagamento {  7 usages  2 inheritors
    protected int id;  1 usage
    protected String nome;  3 usages
    protected String dataNascimento;  1 usage
    protected String endereco;  1 usage
    protected String tipoCliente;  3 usages
    protected boolean ativo;  8 usages
    protected String telefone;  1 usage
    protected StatusPagamento statusPagamento;  6 usages

    public abstract void cadastrar();  6 usages  2 implementations
    public abstract Cliente remover(int id);  2 usages  2 implementations
    public abstract void plano();  3 usages  2 implementations

    @Override  1 override
    public String toString() {
        return "Cliente{" +
                "id=" + id +
                ", nome='" + nome + '\'' +
                ", dataNascimento='" + dataNascimento + '\'' +
                ", endereco='" + endereco + '\'' +
                ", tipoCliente='" + tipoCliente + '\'' +
                ", ativo=" + ativo +
                ", telefone='" + telefone + '\'' +
                ", statusPagamento=" + statusPagamento +
                '}';
    }

    public void setStatusPagamento(StatusPagamento statusPagamento) {  6 usages
        this.statusPagamento = statusPagamento;
    }
```

Figura 5 – Classe Abstrata Cliente.

Fonte: Elaborado pelos autores (2026)

```java
32
33      public void efetuarPagamento() {  4 usages
34          if (statusPagamento != null) {
35              statusPagamento.efetuarPagamento();
36          } else {
37              System.out.println("Status de pagamento não definido.");
38          }
39      }
40
41      public void verificarStatusPagamento() {  8 usages
42          if (statusPagamento != null) {
43              statusPagamento.consultarStatusFinanceiro();
44          } else {
45              System.out.println("Status de pagamento não definido.");
46          }
47      }
48
49      public boolean mudarStatus(int id) {  2 usages
50          this.ativo = !this.ativo;
51          return this.ativo;
52      }
53
54      public void consultarPlanilha() {  no usages
55          System.out.println("Consultando planilha...");
56      }
57  }
```

Figura 6 – Continuação da Classe Abstrata Cliente

Fonte: Elaborado pelos autores (2026).

5.2 Implementação de Herança e Polimorfismo

A Herança permite que as classes Online e Presencial herdem as características da classe Cliente, evitando a repetição de código. O Polimorfismo ocorre quando usamos o mesmo método (consultarPlanilha), mas ele se comporta de formas diferentes dependendo do tipo de aluno. Anulação de Método (@Override): É o ato de reescrever um método da classe pai na classe filha para atender uma necessidade específica.

```java
public class Online extends Cliente {  5 usages
    private float valor;  1 usage
    private String plataforma;  2 usages
    private int tempoAcesso;  1 usage

    @Override
    public String toString() {
        return "Online{" +
                "valor=" + valor +
                ", plataforma='" + plataforma + '\'' +
                ", tempoAcesso=" + tempoAcesso +
                ", id=" + id +
                ", nome='" + nome + '\'' +
                ", dataNascimento='" + dataNascimento + '\'' +
                ", endereco='" + endereco + '\'' +
                ", tipoCliente='" + tipoCliente + '\'' +
                ", ativo=" + ativo +
                ", telefone='" + telefone + '\'' +
                ", statusPagamento=" + statusPagamento +
                '}';
    }

    @Override  6 usages
    public void cadastrar() {
        this.ativo = true;
        this.tipoCliente = "Online";
        System.out.println("Cliente online cadastrado.");
    }

    @Override  2 usages
    public Cliente remover(int id) {
        this.ativo = false;
        System.out.println("Cliente online removido.");
        return this;
    }

```

Figura 7 – Classe Online.

Fonte: Elaborado pelos autores (2026).

```java
    @Override  3 usages
    public void plano() {
        System.out.println("Plano online via: " + plataforma);
    }

    public void enviarFeedbackSemanal(String relatorioTexto, String linkVideo) {  1 usage
        System.out.println("Feedback enviado: " + relatorioTexto);
    }

    @Override  3 usages
    public void consultarStatusFinanceiro() {
        this.verificarStatusPagamento();
    }
}
```

Figura 8 – Continuação de Classe Online.

Fonte: Elaborado pelos autores (2026).

```java
import java.util.List;

public class Presencial extends Cliente {  6 usages
    private String localTreino;  1 usage
    private float frequencia;  no usages
    private int duracaoSessao;  no usages
    private float valor;  no usages
    private List<HorarioTreino> horarios;  no usages

    @Override  6 usages
    public void cadastrar() {
        this.ativo = true;
        this.tipoCliente = "Presencial";
        System.out.println("Cliente presencial cadastrado.");
    }

    @Override  2 usages
    public Cliente remover(int id) {
        this.ativo = false;
        System.out.println("Cliente presencial removido.");
        return this;
    }

    @Override  3 usages
    public void plano() {
        System.out.println("Plano presencial no local: " + localTreino);
    }

    public void registrarDesempenho() {  1 usage
        System.out.println("Desempenho registrado.");
    }

    @Override  3 usages
    public void consultarStatusFinanceiro() {
        this.verificarStatusPagamento();
    }
}
```

Figura 9 – Classe Presencial.

Fonte: Elaborado pelos autores (2026).

5.3 Implementação de Composição e Agregação

As associações entre as classes definem como os objetos se relacionam no mundo real do Tiago:

- Composição: Usada entre Presencial e HorarioTreino. O horário de aula não existe sem o aluno; se o aluno presencial for excluído, sua agenda também desaparece.
- Agregação: Usada entre Treinos e Exercicio_Treino. Um treino possui vários exercícios, mas os exercícios (como "Agachamento") continuam existindo no banco de dados mesmo que o treino seja deletado.

```java
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

public class Treinos {  2 usages
    private int id;  1 usage
    private String nome;  1 usage
    private List<ExercicioTreino> exercicios = new ArrayList<>();  8 usages
    private Cliente cliente;  1 usage
    private Date data;  1 usage
    private boolean status;  2 usages


    @Override
    public String toString() {
        return "Treinos{" +
                "id=" + id +
                ", nome='" + nome + '\'' +
                ", exercicios=" + exercicios +
                ", cliente=" + cliente +
                ", data=" + data +
                ", status=" + status +
                '}';
    }

    public void adicionarExercicio(ExercicioTreino ex) {  1 usage
        exercicios.add(ex);
    }

    public void removerExercicio(int id) {  no usages
        exercicios.removeIf( ExercicioTreino e -> e.equals(id));
    }

    public void adicionarRemovido(ExercicioTreino ex) {  no usages
        exercicios.remove(ex);
    }

```

Figura 10 – Classe Treinos.

Fonte: Elaborado pelos autores (2026).

```
38      public void excluirRemovidos() {  no usages
39          exercicios.clear();
40      }
41
42      public int obterDuracaoEstimada() {  1 usage
43          return exercicios.size() * 5; // exemplo
44      }
45
46      public void finalizarTreino() {  1 usage
47          this.status = true;
48      }
49
50      public float calcularVolumeTotal() {  1 usage
51          float total = 0;
52          for (ExercicioTreino e : exercicios) {
53              total += e.calcularCargaTotal();
54          }
55          return total;
56      }
57
58      public List<ExercicioTreino> listarExerciciosConcluidos() {  no usages
59          List<ExercicioTreino> concluidos = new ArrayList<>();
60          for (ExercicioTreino e : exercicios) {
61              if (e != null) concluidos.add(e);
62          }
63          return concluidos;
64      }
65  }
```

Figura 11 – Continuação da Classe Presencial.

Fonte: Elaborado pelos autores (2026).

```
Rename usages
1  public class ExercicioTreino {
2      private int id;  no usages
3      private Exercicio exercicio;  no usages
4      private String treino;  no usages
5      private float peso;  2 usages
6      private float intensidade;  no usages
7      private int series;  2 usages
8      private int repeticoes;  2 usages
9      private String carga;  no usages
10     private int tempoDescanso;  no usages
11     private boolean status;  1 usage
12
13     public void ajustarVolume(int novasSeries, int novasRepeticoes) {  no usages
14         this.series = novasSeries;
15         this.repeticoes = novasRepeticoes;
16     }
17
18     public void marcarComoConcluido() {  no usages
19         this.status = true;
20     }
21
22     public float calcularCargaTotal() {  no usages
23         return peso * series * repeticoes;
24     }
25
26     public void incrementarCarga(float valor) {  no usages
27         this.peso += valor;
28     }
29 }
```

Figura 12 – Classe ExercicioTreino.

Fonte: Elaborado pelos autores (2026).

```java
public class Exercicio {  3 usages
    private int id;  2 usages
    private String nome;  4 usages
    private String descricao;  4 usages


    @Override
    public String toString() {
        return "Exercicio{" +
                "id=" + id +
                ", nome='" + nome + '\'' +
                ", descricao='" + descricao + '\'' +
                '}';
    }

    public void adicionarExercicio(String nome, String descricao) {  1 usage
        this.nome = nome;
        this.descricao = descricao;
    }

    public void excluirExercicio(int id) {  no usages
        if (this.id == id) {
            this.nome = null;
            this.descricao = null;
        }
    }

    public String obterResumo() {  1 usage
        return nome + ": " + descricao;
    }
}
```

Figura 13 – Classe Exercicio.

Fonte: Elaborado pelos autores (2026).

```java
import java.util.Date;

public class HorarioTreino {  1 usage
    private int id;  no usages
    private Presencial cliente;  no usages
    private Date data;  5 usages
    private boolean checkIn;  2 usages

    public void registrarPresenca(Date dataHora) {  no usages
        this.checkIn = true;
    }

    public void registrarFalta(boolean justificada) {  no usages
        this.checkIn = false;
    }

    public void agendarAula(Date data) {  no usages
        this.data = data;
    }

    public void remarcarAula(Date novaData) {  no usages
        this.data = novaData;
    }

    public void cancelarAula() {  no usages
        this.data = null;
    }

    public String consultarAgenda() {  no usages
        return data != null ? data.toString() : "Sem agendamento";
    }
}
```

Figura 14 – Classe HorarioTreino

Fonte: Elaborado pelos autores (2026).

5.4 Explicação Didática da Lógica

A implementação técnica apresentada acima não é apenas um exercício de sintaxe, mas uma resposta direta às dores operacionais identificadas no cotidiano do personal trainer Tiago. O objetivo central desta lógica é substituir o atual "ar de amadorismo" e a fragmentação de dados por uma estrutura robusta, profissional e escalável.

Abaixo, detalhamos como cada escolha de programação resolve um problema específico do negócio:

- Combate à Ineficiência (O problema da "1 hora"): Ao utilizarmos a Agregação entre Treinos e Exercicio_Treino, permitimos que o Tiago selecione exercícios de uma base pré-existente de forma interativa. Isso ataca diretamente a maior dificuldade do cliente: o gasto excessivo de tempo (cerca de uma hora) para montar um único treino semanal devido à necessidade de preencher manualmente planilhas simples.
- Fim da Fragmentação de Dados: Através da Herança e da Classe Abstrata Cliente, o sistema organiza de forma clara a separação entre alunos Online e Presencial, que antes ficavam dispersos entre Google Sheets e MFIT Personal. Isso centraliza a gestão e elimina a confusão na conciliação dos dois modelos de consultoria.
- Profissionalização Financeira: A implementação da Interface StatusPagamento garante que o sistema possua um controle rigoroso de inadimplência. Como Tiago relatou que o gerenciamento de pagamentos era "praticamente nenhum", a lógica de programação agora permite emitir alertas de renovação e cortar o acesso de alunos inadimplentes, garantindo a saúde financeira do microempreendimento.
- Escalabilidade e Personalização: O uso de Polimorfismo permite que o sistema trate cada aluno conforme suas necessidades específicas (presencial ou online) sem perder a qualidade técnica, permitindo que o profissional gerencie um volume muito maior de clientes simultaneamente.

6 CONSIDERAÇÕES FINAIS

O objetivo geral deste projeto consistiu na aplicação dos fundamentos do paradigma de orientação a objetos para o desenvolvimento de uma solução tecnológica que apoiasse as atividades do personal trainer Tiago. A intenção central foi profissionalizar a gestão e permitir a escalabilidade do atendimento, combatendo diretamente o "ar de amadorismo" causado pela fragmentação de ferramentas e a ineficiência operacional de gastar cerca de uma hora para a criação de um único treino semanal. Através da automação da prescrição e do controle e gerenciamento de alunos, buscou-se transformar um processo manual e disperso em um sistema centralizado e eficiente.

Os procedimentos realizados envolveram o mapeamento das regras de negócio por meio de entrevistas diretas via mensagens e videochamadas, o que permitiu identificar as fricções no fluxo de trabalho do profissional. A partir disso, foram aplicadas ferramentas de diagnóstico como a Matriz SWOT e o Plano de Ação 2W1H, que serviram de base para a modelagem do Diagrama de Classes UML. A implementação técnica em Java utilizou recursos avançados como herança, polimorfismo e interfaces para garantir que o software atendesse aos requisitos de gerenciar alunos presenciais e online, além de automatizar a escrita dos treinos e fornecer relatórios financeiros.

A atividade apresentou algumas fragilidades, como a coleta de dados realizada exclusivamente de forma remota, o que pode ter limitado a compreensão de nuances da rotina prática de Tiago nas academias. Além disso, o escopo atual foca apenas na gestão interna, deixando a interação direta do aluno com a plataforma como uma lacuna a ser preenchida. Outro ponto crítico é a dependência da qualidade do relato pessoal do cliente sobre os equipamentos disponíveis nas diferentes academias onde atua, o que influencia diretamente na precisão da montagem dos treinos.

7 CONTRIBUIÇÕES DA UCE PARA A FORMAÇÃO DISCENTE

A participação nesta Unidade Curricular de Extensão (UCE) demonstra ser fundamental para a transição do conhecimento acadêmico para a prática profissional, cumprindo o objetivo central de aplicar os fundamentos da Programação Orientada a Objetos (POO) em um contexto real. Para o grupo, o desenvolvimento do sistema para o personal trainer Tiago permitiu a consolidação de conceitos técnicos complexos, como herança, polimorfismo e interfaces, ao traduzir necessidades reais de negócio em uma arquitetura de software funcional. A exigência de resolver a ineficiência operacional de um profissional autônomo forçou a equipe a ir além da sintaxe de programação, focando na criação de soluções que realmente impactem a produtividade e a escalabilidade do cliente.

No parecer do grupo, a atividade transcendeu o âmbito técnico, estimulando a consciência cidadã e a ética profissional ao lidar diretamente com as dificuldades de um microempreendedor da comunidade local. Ao diagnosticar problemas críticos como a fragmentação de dados e o risco de inadimplência, foi possível vivenciar a responsabilidade social do desenvolvedor em promover o crescimento econômico e a inovação tecnológica através do software. Essa experiência de interação e socialização foi essencial para desenvolver habilidades de comunicação e levantamento de requisitos que não seriam exercitadas em cenários puramente teóricos de sala de aula.

Por fim, a correlação direta do projeto com os Objetivos de Desenvolvimento Sustentável (ODS), especificamente as ODS 8, 9 e 11, reforça a relevância desta atividade para uma formação acadêmica humanizada e consciente. O desafio de projetar uma ferramenta multiplataforma que reduza drasticamente o tempo de trabalho do Tiago nos prepara para as exigências reais do mercado de trabalho, onde a agilidade e a entrega de valor tecnológico são fundamentais. Dessa forma, a UCE contribui para formar profissionais que não apenas dominam paradigmas de programação, mas que compreendem a tecnologia como um motor de transformação social e fortalecimento da infraestrutura de pequenos negócios em seu entorno.